Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

LABORATORIO DE INVESTIGACIÓN EPIDEMIOLÓGICA DE GUATEMALA

202400173 – José Paúl Lorenty Herrera

Resumen

El presente proyecto consiste en el desarrollo de una aplicación capaz de simular el comportamiento de enfermedades infecciosas en tejidos celulares representados mediante rejillas cuadradas. A partir de un patrón inicial de células sanas y contagiadas, el sistema aplica reglas específicas que permiten observar cómo evoluciona la infección a lo largo de distintos períodos, con el objetivo de identificar patrones repetitivos que permitan determinar si la enfermedad será leve, grave o mortal para un paciente.

La solución fue desarrollada utilizando el lenguaje de programación C# y el paradigma de Programación Orientada a Objetos. Además, se implementaron estructuras de datos basadas en listas enlazadas para gestionar la información de pacientes, filas y celdas dentro de cada rejilla. El programa también permite cargar datos desde archivos XML y generar representaciones visuales de las estructuras mediante Graphviz.

Como resultado, el sistema facilita el análisis del comportamiento de patrones celulares y demuestra la utilidad de las simulaciones computacionales en el estudio de sistemas complejos.

Palabras clave

I. *Listas*
II. *Nodos*
III. *Programación*
IV. *Sistema*

Abstract

This project consists of the development of an application capable of simulating the behavior of infectious diseases in cellular tissues represented through square grids. Starting from an initial pattern of healthy and infected cells, the system applies specific rules that allow the observation of how the infection evolves over different periods, with the objective of identifying repeating patterns that determine whether the disease will be mild, severe, or fatal for a patient.

The solution was developed using the C# programming language and the Object-Oriented Programming (OOP) paradigm. In addition, data structures based on linked lists were implemented to manage patient information, rows, and cells within each grid. The program also allows data to be loaded from XML files and generates visual representations of the structures using Graphviz.

As a result, the system facilitates the analysis of cellular pattern behavior and demonstrates the usefulness of computational simulations in the study of complex systems.

Keywords

I. *Lists*
II. *Nodes*
III. *Programming*
IV. *System*

Introducción

El presente proyecto describe el desarrollo de una aplicación orientada a la simulación del comportamiento de enfermedades en tejidos celulares, representados mediante rejillas o matrices. Este tipo de simulaciones es relevante en el ámbito científico y tecnológico, ya que permite analizar patrones de propagación y evolución de enfermedades a partir de reglas específicas aplicadas a cada célula del sistema. En este proyecto se implementa una solución utilizando el lenguaje C# y el paradigma de Programación Orientada a Objetos (POO), incorporando además estructuras de datos abstractas como listas enlazadas para el manejo eficiente de la información de pacientes y de las células que conforman cada rejilla. La aplicación permite cargar datos desde archivos XML, ejecutar simulaciones por períodos y visualizar gráficamente los resultados mediante herramientas de generación de diagramas, como lo es graphviz. El proyecto demuestra la importancia del uso de estructuras de datos y simulaciones computacionales para el análisis de patrones.

C#

C# es un lenguaje de programación moderno, innovador, de código abierto, multiplataforma orientado a objetos y uno de los 5 principales lenguajes de programación de GitHub

.NET

Es un conjunto de herramientas, tecnologías y servicios que dan pie a la creación de distintas aplicaciones, es entonces, una plataforma de desarrollo de software. ¿Plataforma? Esto se refiere al conjunto de elementos que sirven para “construir algo”. Para este caso ayudan a hacer funcionar aplicaciones web, de escritorio, de consola, entre otros.

POO (Programación orientada a objetos)

Es un paradigma de Programación. Donde el código se organiza alrededor de objetos que pueden contener datos (atributos) y procedimientos (métodos) que operan sobre esos datos. Se centra en modelar entidades del mundo real como objetos con comportamientos y propiedades.

Objetos

Elemento que representa cualquier cosa. Son instancias concretas de una clase. Cada objeto tiene un estado (valores de sus atributos) y un comportamiento (métodos que puede ejecutar).

Atributos

Los atributos son variables que contienen los datos asociados a un objeto. Cada objeto tiene su propio conjunto de atributos, que representan su estado en un momento dado.

Métodos

Los métodos son funciones o procedimientos que definen el comportamiento de un objeto. Los métodos pueden acceder y modificar los atributos de un objeto, así como realizar otras acciones relacionadas con el objeto.

Clases

Pueden definirse las clases como “moldes” o “plantillas” en donde se define la estructura y comportamiento que tendrán los objetos. Se describe lo que son los atributos y métodos que los objetos tendrán.

Encapsulamiento

El encapsulamiento es el principio de ocultar los detalles internos de un objeto y exponer solo una interfaz pública para interactuar con él.

Abstracción

La abstracción es el proceso de identificar las características esenciales de un objeto e ignorar los detalles no relevantes.

Herencia

La herencia es un mecanismo que permite que una clase herede atributos y métodos de otra clase, facilitando la reutilización de código y la creación de jerarquías de clases.

Polimorismo

El polimorfismo es la capacidad de un objeto de comportarse de diferentes maneras según el contexto.

XML

Es un metalenguaje que fue diseñado para estructurar, almacenar e intercambiar datos entre aplicaciones. Es un estándar ya que es extensible y puede ser utilizado independientemente de la plataforma.

Partes de un XML

- Prólogo : En este se especifica que el documento en cuestión es XML, se define la versión y la codificación de caracteres que se utilizó en el archivo.
- Elementos: Se tiene una etiqueta o tag inicial < > y una final </>.
- Epílogo: Datos adicionales como encriptación. Generalmente los tokens son agregados en este apartado. Este elemento es opcional.

Estructura de Datos

Son aquellas que nos permiten, como desarrolladores, organizar la información de manera eficiente, y en definitiva diseñar la solución correcta para un determinado problema. Ya sean las más utilizadas comúnmente, como las variables, arrays, conjuntos o clases o las diseñadas para un propósito específico como los árboles, grafos tablas, etc.

TDA's

Un tipo abstracto de datos es una colección de valores y de operaciones que se definen mediante una especificación independiente de cualquier representación. Se enfoca en qué operaciones se pueden realizar, no en cómo están implementadas. Implementar un TDA significa elegir una representación para el conjunto de valores del tipo de datos, así como codificar sus operaciones en un lenguaje de programación. Es posible tener varias implementaciones para un mismo TAD, pero el comportamiento debe ser siempre el mismo.

Tipos de TDA's

- Estaticas : Son aquellas en las que el tamaño ocupado en memoria se define antes de que el programa se ejecute y no puede modificarse dicho tamaño durante la ejecución del programa.
- Dinamicas: Son aquellas en la que el tamaño ocupado en memoria puede modificarse durante la ejecución del programa.

Estructura de Datos lineales

Las estructuras de datos lineales son aquellas en las que los elementos ocupan lugares sucesivos en la estructura y cada uno de ellos tiene un único sucesor y un único predecesor, es decir, sus elementos están ubicados un al lado del otro relacionados en forma lineal.

Nodos

Un nodo es un registro que contiene un dato de interés y al menos un puntero para referenciar (apuntar) a otro nodo. Si la estructura tiene solo un puntero, la única estructura que se puede construir con él es una lista, y si el nodo tiene más de un puntero se pueden construir estructuras más complejas como árboles o grafos.

Apuntadores

Es una variable que ayuda a llevar el control de una estructura. De su manera más básica se puede describir como una dirección de memoria física en donde se almacena un dato. En este caso, en una estructura de datos, el apuntador almacena una dirección de memoria de cierta posición importante de la estructura.

Listas

Al igual que una cadena, una lista es una secuencia de valores. En una cadena, los valores son caracteres; en una lista, pueden ser de cualquier tipo, y estos valores se llaman elementos. En C# existen dos formas comunes de trabajar con listas: las colecciones incluidas en .NET (como List) y las listas que se pueden implementar mediante clases, por ejemplo usando estructuras como listas enlazadas siguiendo el paradigma de la programación orientada a objetos.

Listas Enlazadas

En las estructuras de datos, las listas enlazadas se construyen con elementos que están ubicados en una secuencia. Aquí, cada elemento se conecta con el siguiente a través de un enlace que contiene la posición del siguiente elemento. De este modo, teniendo la referencia del principio de la lista podemos acceder a todos los elementos de la misma.

Listas simplemente enlazadas

Una lista doblemente enlazada es una estructura de datos formada por un conjunto de nodos enlazados secuencialmente, donde cada nodo contiene datos y dos referencias: una al siguiente nodo y otra al anterior. Esto permite recorrer la lista en ambos sentidos (hacia adelante y hacia atrás), facilitando operaciones como inserción y eliminación de elementos en cualquier posición de la lista.

Listas doblemente enlazadas

Una lista doblemente enlazada es una estructura de datos formada por un conjunto de nodos enlazados secuencialmente, donde cada nodo contiene datos y dos referencias: una al siguiente nodo y otra al anterior. Esto permite recorrer la lista en ambos sentidos (hacia adelante y hacia atrás), facilitando operaciones como inserción y eliminación de elementos en cualquier posición de la lista.

Graphviz

Graphviz es un software de visualización de gráficos de código abierto. La visualización de gráficos es una forma de representar información estructural como diagramas de gráficos abstractos y redes. Los programas de diseño Graphviz toman descripciones de gráficos en un lenguaje de texto simple y crean diagramas en formatos útiles, como imágenes y SVG.

Lenguaje DOT

El lenguaje DOT es un formato de texto simple para describir grafos. Un grafo es una estructura que consiste en nodos (puntos o vértices) conectados por aristas (líneas o edges).

Lectura y Escritura

Leer un archivo XML significa interpretar su contenido para obtener información almacenadaen forma de elementos, atributos y texto. Durante la lectura, una aplicación recorre laestructura jerárquica del documento y extrae los datos que necesita, por ejemplo, el valor deuna etiqueta como <nombre> o <edad>.

Conclusiones

- El desarrollo de este proyecto permitió demostrar la importancia de aplicar estructuras de datos y principios de programación orientada a objetos para resolver problemas complejos mediante simulaciones computacionales. La implementación de listas enlazadas para el manejo de pacientes, filas y celdas permitió gestionar eficientemente la información

necesaria para representar y analizar las rejillas de tejido celular. Asimismo, la simulación de los distintos períodos permitió identificar patrones repetitivos que facilitan determinar el comportamiento de una enfermedad dentro del sistema analizado. Esto evidencia cómo los algoritmos y las estructuras de datos pueden utilizarse para modelar fenómenos reales, permitiendo comprender de manera más clara la evolución de ciertos procesos biológicos mediante herramientas informáticas.

- El uso de archivos XML para el ingreso y generación de datos, junto con la visualización mediante Graphviz, permitió complementar el análisis técnico con representaciones gráficas que facilitan la interpretación de los resultados obtenidos. Este tipo de herramientas demuestra el valor de integrar diferentes tecnologías dentro de una misma solución de software. Como recomendación, futuras mejoras podrían enfocarse en optimizar el rendimiento para manejar rejillas de mayor tamaño o en incorporar nuevas formas de visualización que permitan analizar con mayor profundidad los patrones generados en cada simulación.

Referencias bibliográficas

Azul School. (s. f.). *¿Qué es .NET y para qué sirve?* https://azulschool.net/que-esnet-y-para-que-sirve/

CEV. (s. f.). *4 pilares de la programación orientada a objetos.* https://www.cev.com/blog/4-pilares-de-la-programacion-orientada-a-objetos

Graphviz. (s. f.). *Graphviz documentation.* https://graphviz.readthedocs.io/en/stable/

Kodigo.org. (s. f.). *Lenguaje C# — Qué es y para qué sirve.* https://kodigo.org/lenguaje-c-c-sharp-que-es-y-para-que-sirv

Microsoft. (2023). *LINQ to XML overview.* Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/standard/linq/linq-to-xml-overview

TechTarget. (s. f.). *Data structure.* https://www.techtarget.com/searchdatamanagement/definition/data-structure

W3Schools. (s. f.). *Linked lists types.* https://www.w3schools.com/dsa/dsa_data_linkedlists_types.php

Anexos

Release/semana	Actividad
Primer release/ Semana 1	Versión del sistema con el menú en consola para interactuar con las funciones.
Segundo reléase/ Semana 2	Se agregó el algoritmo de infección con base a lo establecido en el enunciado.
Tercer release/ Semana 3	Se agregaron y merjoraron las funcionalidades relacionadas con los archivos .xml y se probaron archivos de prueba.
Cuarto Release / semana 4	Se mejoró la estética de la ejecución además de agregar la documentación.

Tabla 1 Cronograma de actividades

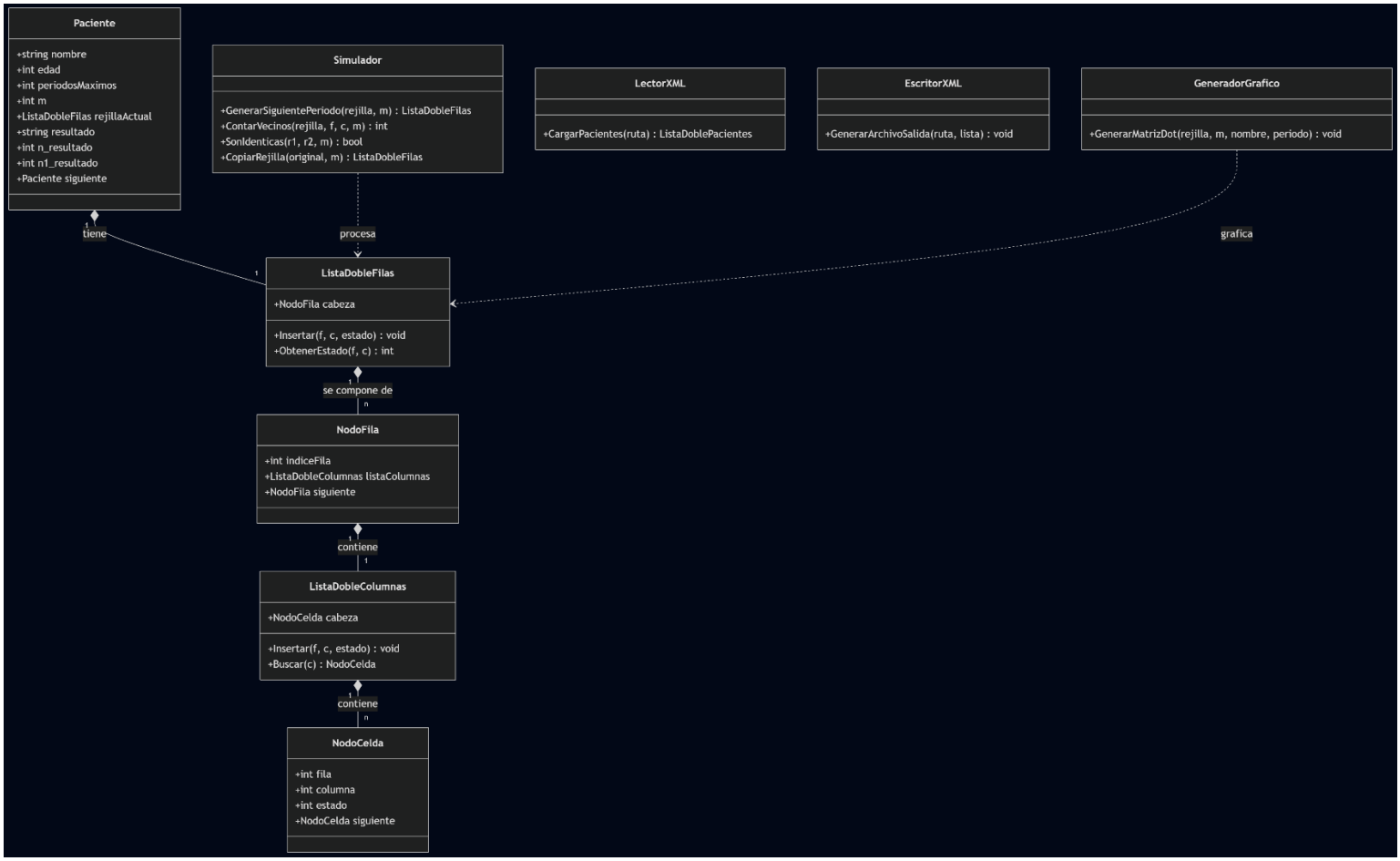

Ilustración 1 Diagrama de Clases

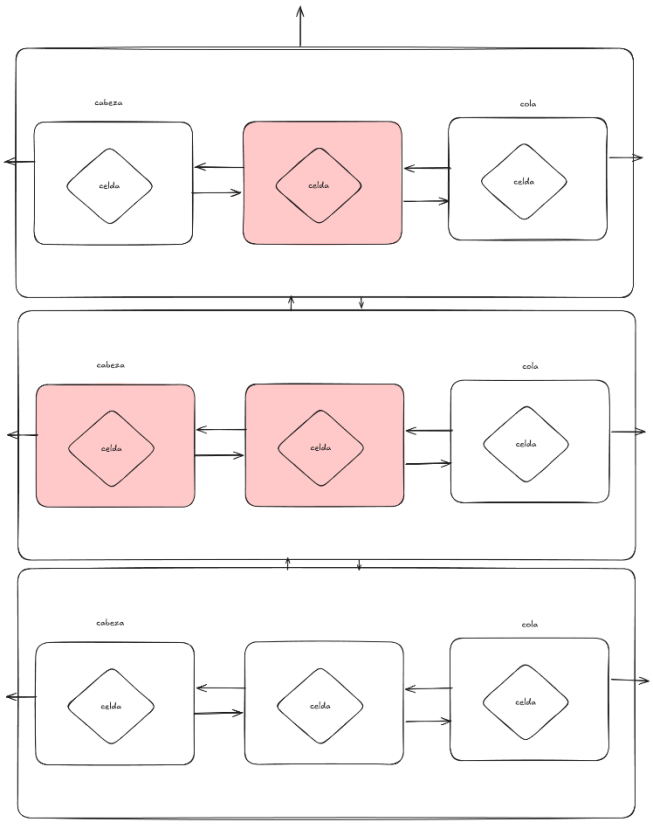

Ilustración 2 Diagrama de Rejillas

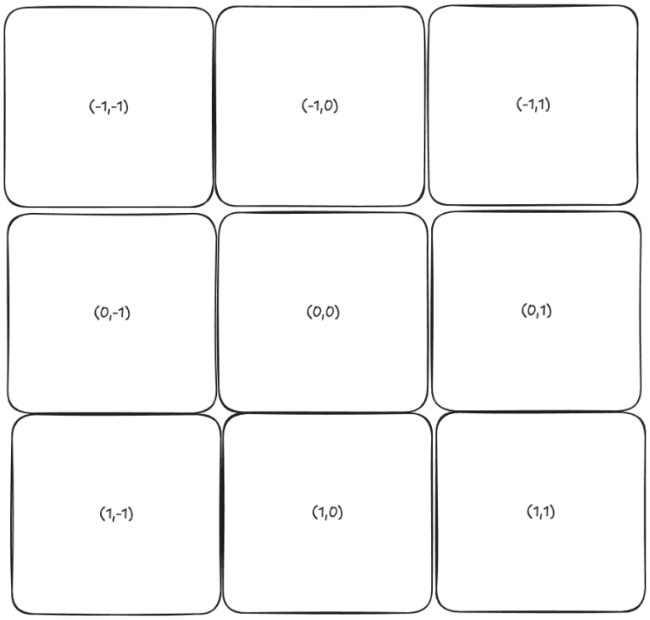

Ilustración 3 Diagrama de Vecinos